REAL-TIME AIR QUALITY MONITORING AND POLLUTION PREDICTION

PROJECT REPORT

Submitted by:

GROUP - 2

RAHUL B - (CB.SC.U4AIE23114)

KARTHIKEYA CH - (CB.SC.U4AIE23121)

MOHAN RAJ S - (CB.SC.U4AIE23147)

CHARAN TEJA G - (CB.SC.U4AIE23172)

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE

AMRITA VISHWA VIDYAPEETHAM

COIMBATORE - 641112, INDIA

March 2026

AMRITA SCHOOL OF ARTIFICIAL INTELLIGENCE
COIMBATORE – 641112

BONAFIDE CERTIFICATE

This is to certify that the report entitled **"Real-Time Air Quality Monitoring and Pollution Prediction"** submitted by Rahul B (CB.SC.U4AIE23114), Karthikeya CH (CB.SC.U4AIE23121), Mohan Raj S (CB.SC.U4AIE23147), and Charan Teja G (CB.SC.U4AIE23172) for the award of the Degree of Bachelor of Technology in **"COMPUTER SCIENCE ENGINEERING - ARTIFICIAL INTELLIGENCE"** is a bonafide record of the work carried out under my guidance and supervision at Amrita School of Artificial Intelligence, Coimbatore.

Project Guide : Dr. SREEJA B P
Assistant Professor (Sr. Gr.)

Submitted for the University Examination held on 10/03/26

Contents

Abstract **4**

1 Introduction **5**

1.1 Background and Motivation 6

1.2 Problem Statement 6

1.3 Scope and Limitations 6

2 Literature Review **7**

2.1 Overview of Existing Work 7

2.2 Frameworks, Algorithms, and Tools 8

2.3 Research Gap 8

3 Dataset Description **9**

3.1 Source of Data 9

3.2 Dataset Characteristics 9

3.3 Preprocessing Steps 10

4 Architecture & Tools **11**

4.1 Big Data Ecosystem Components 11

4.2 System Architecture 11

4.3 Architecture Diagram 12

4.4 Justification for Selected Tools 12

5 Methodology **13**

5.1 Data Collection 13

5.2 Data Preprocessing 13

5.3 Feature Engineering 14

5.4 Model Training 14
5.5 Evaluation and Storage 14
5.6 Implementation 15
5.6.1 Technologies Used 16
5.6.2 System Requirements 16

6 Results **17**
6.1 Dataset Storage in HDFS 17
6.2 Preprocessed Dataset 18
6.3 Saved Machine Learning Models 19
6.4 Model Performance Visualization 20

7 Discussion **21**
7.1 Interpretation of Results 21
7.2 Challenges Faced 21
7.3 Lessons Learned 21

8 Conclusion and Future Work **22**
8.1 Conclusion 22
8.2 Future Work 22

9 References **23**

10 Appendices **24**

Abstract

Air pollution has become one of the most critical environmental and public health challenges worldwide. Continuous monitoring and accurate prediction of air quality are essential for taking preventive measures and improving environmental policies. With the rapid growth of IoT sensors and environmental monitoring systems, large volumes of air quality data are generated continuously, making traditional data processing techniques inefficient.

This project presents a scalable Big Data analytics framework for real-time air quality monitoring and pollution prediction. The system utilizes Apache Hadoop Distributed File System (HDFS) for distributed storage and Apache Spark for large-scale data processing and analysis. A large air pollution dataset containing pollutant concentrations such as PM2.5, PM10, NO_2, SO_2, CO, and O_3 along with environmental parameters like temperature, humidity, and wind speed is used for the study.

Data preprocessing and feature engineering are performed using Spark DataFrames and RDDs to handle missing values and prepare the dataset for machine learning. A Random Forest regression model implemented using Spark MLlib is applied to predict pollutant concentration levels. The distributed architecture enables efficient handling of large-scale datasets and significantly improves processing speed and scalability.

The results demonstrate that the proposed system can effectively analyze air quality trends and generate accurate pollution predictions. The system also provides visualization of air quality metrics, enabling better understanding of pollution patterns and supporting data-driven decision making for environmental monitoring and public health management.

1. Introduction

Air pollution has become one of the most serious environmental challenges affecting human health and ecological sustainability across the world. Rapid urbanization, industrial activities, and increasing vehicle emissions have significantly contributed to the rise of harmful pollutants in the atmosphere. Pollutants such as PM2.5, PM10, nitrogen dioxide (NO_2), sulfur dioxide (SO_2), carbon monoxide (CO), and ozone (O_3) can cause severe health problems including respiratory diseases, cardiovascular disorders, and other long-term health effects.

With the growth of smart cities and environmental monitoring systems, large volumes of air quality data are continuously generated from sensors and monitoring stations. These datasets contain high volume, velocity, and variety of information, making them difficult to process using traditional data processing techniques. Big Data technologies provide scalable and efficient solutions to handle such massive datasets through distributed storage and parallel processing.

This project focuses on developing a scalable system for monitoring and predicting air pollution levels using Big Data analytics techniques. The system utilizes Apache Hadoop Distributed File System (HDFS) for storing large-scale environmental datasets and Apache Spark for efficient data processing and analysis. Machine learning techniques, specifically Random Forest regression using Spark MLlib, are applied to analyze historical air quality data and predict future pollution levels.

The proposed system demonstrates how Big Data frameworks can effectively process large environmental datasets, identify pollution trends, and provide accurate predictions. Such systems can help governments and environmental agencies make informed decisions for pollution control, public health protection, and sustainable urban development.

1.1 Background and Motivation

Air quality monitoring is essential for understanding environmental conditions and protecting public health. Traditional monitoring systems generate large volumes of data that need efficient storage and processing techniques. Big Data technologies such as Hadoop and Spark allow distributed processing of massive datasets, enabling faster analysis and better scalability for environmental monitoring applications.

1.2 Problem Statement

The major challenge in air quality monitoring is handling large-scale environmental datasets and predicting future pollution levels accurately. Traditional data processing systems are not capable of efficiently managing high-volume sensor data. Therefore, there is a need for a scalable Big Data framework that can store, process, and analyze air quality data using distributed computing technologies and machine learning models.

1.3 Scope and Limitations

The scope of this project is to develop a Big Data analytics framework for air quality monitoring and pollution prediction. The system uses Hadoop for distributed storage and Apache Spark for large-scale data processing and machine learning analysis. Historical air quality datasets containing pollutant concentrations and environmental parameters are used to train prediction models.

However, the project has certain limitations. Accessing real-time environmental data through APIs was challenging due to reliability and response issues. Additionally, some regional datasets were not properly structured, which required additional preprocessing and the use of alternative datasets. Despite these challenges, the system demonstrates the effectiveness of Big Data technologies for environmental data analysis and pollution prediction.

2. Literature Review

Several studies have been conducted on air quality monitoring and prediction using data-driven and machine learning approaches. Kang et al. (2018), in their paper "Air Quality Prediction: Big Data and Machine Learning Approaches," reviewed various machine learning and big data techniques used for Air Quality Index (AQI) prediction. Their study highlighted the growing importance of Big Data platforms in handling large-scale environmental datasets generated by monitoring stations and sensor networks. The authors emphasized that distributed computing frameworks can efficiently process large volumes of air quality data and enable scalable real-time environmental monitoring systems.

Bosco and Kowsalya (2024), in their research titled "A Novel Approach for Air Pollution Prediction Using Machine Learning Techniques," implemented machine learning algorithms such as Logistic Regression and K-Nearest Neighbors (KNN) for AQI prediction. Their work demonstrated that proper data preprocessing, feature selection, and normalization significantly improve the accuracy of prediction models. The study also highlighted the importance of machine learning techniques in identifying pollution patterns and improving environmental prediction systems.

2.1 Overview of Existing Work

Air quality prediction systems have evolved significantly with the advancement of data analytics and machine learning technologies. Early approaches mainly relied on statistical models to analyze pollution patterns. However, with the increasing availability of environmental data from monitoring stations and sensor networks, machine learning techniques have become more effective for identifying patterns and predicting pollution levels.

Recent studies have shown that combining machine learning models with big data technologies can significantly improve prediction accuracy and scalability. Distributed

computing platforms allow researchers to process large environmental datasets efficiently and perform complex analytical operations within a shorter time.

2.2 Frameworks, Algorithms, and Tools

Several big data frameworks and machine learning tools are widely used in air quality prediction research. Apache Hadoop is commonly used for distributed data storage through the Hadoop Distributed File System (HDFS), which allows large datasets to be stored across multiple nodes. Apache Spark is widely used for large-scale data processing due to its in-memory computation and parallel processing capabilities.

Machine learning algorithms such as Decision Tree, Random Forest, and Artificial Neural Networks are commonly applied for predicting air pollution levels. Among these algorithms, Random Forest has been widely adopted due to its robustness and ability to handle complex relationships between environmental variables. Tools such as PySpark and Spark MLlib enable scalable machine learning model training and prediction on large environmental datasets.

2.3 Research Gap

Although many research studies have explored machine learning techniques for air pollution prediction, several challenges still exist. Many traditional models struggle to process large-scale environmental datasets efficiently, especially when data is collected from multiple monitoring stations over long periods.

Additionally, several existing systems focus only on localized datasets and do not fully utilize distributed computing frameworks for large-scale environmental data analysis. Therefore, there is a need for scalable big data architectures that can efficiently process large air quality datasets while applying machine learning techniques for accurate pollution prediction.

The proposed system addresses this challenge by implementing a scalable framework using Apache Hadoop for distributed storage and Apache Spark for large-scale data processing and machine learning-based pollution prediction.

3. Dataset Description

3.1 Source of Data

The dataset used in this project is the *Air Pollution in China (2015–2025)* dataset obtained from the Kaggle dataset repository. This dataset contains large-scale historical air quality measurements collected from multiple monitoring stations located across different cities in China. The data was originally gathered from environmental monitoring networks that continuously record pollutant concentrations and meteorological conditions.

The dataset includes several key air pollutants such as PM2.5, PM10, nitrogen dioxide (NO_2), sulfur dioxide (SO_2), carbon monoxide (CO), and ozone (O_3), along with environmental parameters such as temperature, humidity, wind speed, and timestamp information. These attributes provide valuable information for analyzing pollution trends and understanding the factors that influence air quality variations.

The dataset is stored in *Parquet format*, which is a columnar storage format designed for efficient processing of large datasets in big data environments. Parquet provides better compression and faster query performance compared to traditional formats, making it suitable for distributed data processing using frameworks such as Apache Spark and Hadoop. The availability of large-scale time-series environmental data makes this dataset appropriate for applying big data analytics and machine learning techniques for pollution prediction.

3.2 Dataset Characteristics

The dataset contains attributes representing air pollutant concentrations and environmental conditions that influence air quality. The major pollutants included are particulate matter (PM2.5 and PM10), nitrogen dioxide (NO_2), sulfur dioxide (SO_2), carbon monoxide (CO), and ozone (O_3). In addition to pollutant levels, meteorological attributes

such as temperature, humidity, wind speed, and timestamp information are also included.

The dataset is stored in *Parquet format*, a columnar storage format that is optimized for large-scale data processing frameworks such as Apache Spark. Parquet provides efficient data compression and faster query performance, making it well suited for big data analytics tasks. The dataset contains time-series records collected from different monitoring stations, where each record represents pollutant measurements and environmental parameters at a specific time instance.

The total dataset size is approximately 4 GB, which makes it suitable for distributed storage and processing using big data frameworks such as Hadoop and Apache Spark. The dataset also exhibits key Big Data characteristics such as high volume and variety, as it contains large amounts of environmental data with multiple attributes collected over a long period of time.

3.3 Preprocessing Steps

Before applying machine learning models, the dataset was preprocessed to improve data quality and consistency. Data cleaning techniques were used to identify and handle missing or invalid values. In cases where pollutant measurements were incomplete, the corresponding records were either removed or replaced using suitable preprocessing methods.

The timestamp attribute was processed to extract useful temporal features such as hour and day, which help the model understand time-based pollution patterns. Feature normalization and transformation were also applied to maintain consistency among different variables.

The processed dataset was then stored in the Hadoop Distributed File System (HDFS) for distributed storage and parallel processing. Apache Spark DataFrames and RDDs were used to perform large-scale preprocessing operations efficiently. These preprocessing steps ensured that the dataset was properly structured and ready for machine learning model training and pollution prediction.

4. Architecture & Tools

4.1 Big Data Ecosystem Components

The proposed system uses components from the Big Data ecosystem to process and analyze large-scale air quality datasets. The main technologies used include Apache Hadoop, Apache Spark, and Spark MLlib.

Apache Hadoop provides distributed storage through the Hadoop Distributed File System (HDFS), enabling large datasets to be stored and managed across multiple nodes. Apache Spark is used as the primary processing engine for large-scale data analysis and machine learning tasks due to its fast in-memory computation capabilities. Spark MLlib is used to implement machine learning algorithms such as Random Forest regression for predicting pollutant concentrations. Visualization tools such as Streamlit or Power BI are used to analyze and display air quality trends.

4.2 System Architecture

The system architecture consists of several stages including data ingestion, storage, preprocessing, model training, prediction, and visualization. Air quality data from the OpenAQ repository stored in AWS S3 is first uploaded to the Hadoop Distributed File System (HDFS) for distributed storage.

Apache Spark is then used to preprocess the dataset by handling missing values and extracting time-based features. The dataset is divided into training and testing sets, and a Random Forest regression model is trained using Spark MLlib to predict pollutant concentrations. The model performance is evaluated using metrics such as R^2 and RMSE.

Finally, the trained models are stored for future use, and the prediction results are visualized using tools such as Streamlit or Power BI to analyze air quality trends.

4.3 Architecture Diagram

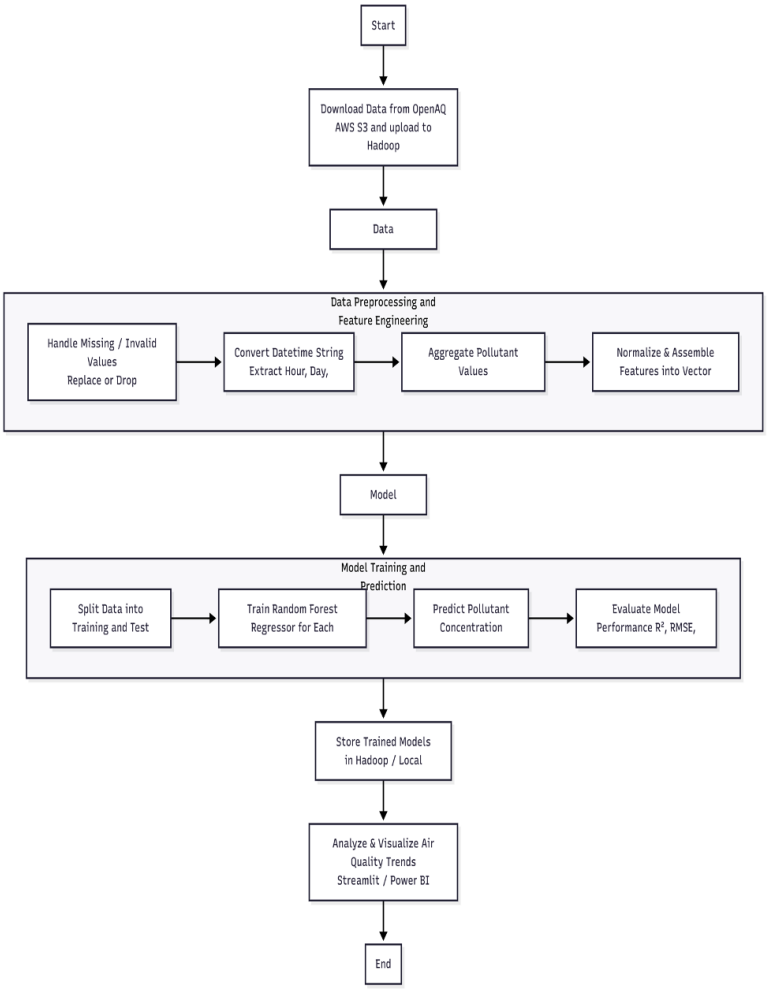

Figure 4.1: System Architecture for Air Quality Monitoring and Pollution Prediction

4.4 Justification for Selected Tools

The tools used in this project were selected for their efficiency in processing large-scale environmental datasets. Apache Hadoop provides distributed storage through HDFS, while Apache Spark enables fast data processing using in-memory computation. Spark MLlib supports scalable machine learning, and the Random Forest algorithm was chosen for its strong performance on large datasets. Visualization tools such as Streamlit and Power BI present the prediction results in an interactive format.

5. Methodology

The proposed air quality monitoring and prediction system follows a structured workflow consisting of several stages including data collection, preprocessing, feature engineering, model training, evaluation, and storage. These stages ensure efficient processing of large-scale environmental datasets while maintaining prediction accuracy.

5.1 Data Collection

The air quality dataset used in this project is obtained from OpenAQ, an open-access platform that provides historical and real-time air quality data from monitoring stations across different regions. The dataset includes pollutant measurements such as PM2.5, PM10, NO_2, SO_2, CO, and O_3, along with temporal information.

Due to limitations in API response time and reliability, the system primarily focuses on historical dataset ingestion. The collected dataset is stored in the Hadoop Distributed File System (HDFS), which enables distributed access and efficient handling of large volumes of environmental data.

5.2 Data Preprocessing

The preprocessing stage is implemented using Apache Spark, which enables parallel and distributed operations on large datasets. Several preprocessing steps are performed to prepare the dataset for analysis:

- **Handling Missing Values:** Missing or null entries in pollutant measurements are handled using forward-fill techniques to maintain data continuity.
- **Timestamp Conversion:** The datetime attribute is converted into a proper timestamp format and sorted to maintain chronological order.
- **Feature Extraction:** Time-based attributes such as hour, day, and month are

extracted from the timestamp to capture temporal patterns in pollution levels.

- **Outlier Handling:** Extreme or erroneous pollutant values are limited within an acceptable range to reduce noise in the dataset.
- **Normalization:** Pollutant concentrations are scaled to maintain a consistent range for model training.

5.3 Feature Engineering

Feature engineering is performed to enhance the predictive capability of the model by creating additional meaningful attributes.

- **Spatial Features:** Each monitoring station is identified using a unique location identifier, allowing the model to learn regional pollution characteristics.
- **Temporal Features:** Time-based features such as hour of the day help identify daily pollution patterns influenced by traffic and industrial activity.
- **Correlation Analysis:** Relationships between different pollutants are analyzed to understand how variations in one pollutant influence others.

5.4 Model Training

The prediction model is implemented using the Random Forest Regressor from Spark MLlib. Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their outputs to improve prediction accuracy.

Each pollutant (PM2.5, PM10, O_3, CO, NO_2, and SO_2) is modeled separately, where the objective is to predict the next-hour concentration based on historical readings and extracted features.

The dataset is divided into training and testing sets using an 80:20 ratio. The training dataset is used to build the model, while the testing dataset is used to evaluate the prediction performance.

5.5 Evaluation and Storage

The trained models are evaluated using performance metrics such as the coefficient of determination (R^2) and Root Mean Square Error (RMSE). These metrics help measure the accuracy and reliability of the prediction models.

Once the models are trained and evaluated, they are stored in the Hadoop Distributed File System (HDFS) for future retrieval and deployment. This enables the system to reuse trained models for predicting air pollution levels and analyzing air quality trends.

5.6 Implementation

The implementation of the proposed air quality monitoring and prediction system integrates multiple Big Data technologies to efficiently process large-scale environmental datasets. The system was designed to handle large volumes of air quality data while maintaining scalability, reliability, and processing efficiency.

The primary data source used in this project is the OpenAQ platform, which provides publicly available air quality data collected from monitoring stations across different locations. Due to limitations in the OpenAQ API performance and response time, the dataset was downloaded directly from AWS S3 using the AWS Command Line Interface (CLI). This method enabled reliable data acquisition and secure transfer of large datasets from the cloud storage environment.

Once the dataset was obtained, it was stored in the Hadoop Distributed File System (HDFS), which served as the main storage layer of the system. HDFS provides distributed storage and allows large datasets to be stored across multiple nodes while ensuring fault tolerance and high availability. This storage mechanism also enables efficient data access for distributed data processing frameworks.

Apache Spark was used as the core data processing engine in the system. Spark was responsible for performing data preprocessing, transformation, and feature extraction operations on the dataset. Spark DataFrames and Spark SQL were utilized to perform distributed data operations efficiently. Tasks such as handling missing values, timestamp processing, and feature extraction were executed using Spark's distributed computing capabilities.

For the machine learning stage, Spark MLlib was used to implement the Random Forest regression model for predicting pollutant concentrations. Random Forest was selected because of its ability to handle large datasets and capture complex relationships between environmental variables. The model training process involved splitting the dataset into training and testing sets, building the prediction model, and evaluating its performance using statistical metrics.

Model evaluation was performed using performance metrics such as the coefficient of determination (R^2) and Mean Absolute Error (MAE). These metrics helped measure the prediction accuracy and reliability of the trained models. After training and evaluation, the models were stored in HDFS so they could be reused for future prediction tasks.

Scala was used as the primary programming language due to its strong compatibility with Apache Spark and its efficient functional programming capabilities. The development environment used for implementation was IntelliJ IDEA, which provided an integrated platform for coding, debugging, and project management. Spark Shell was also used for testing and validating individual data processing operations during development.

5.6.1 Technologies Used

- Programming Language: Scala
- Big Data Framework: Apache Spark (RDDs, DataFrames, MLlib)
- Storage System: Hadoop Distributed File System (HDFS)
- Tools: AWS CLI, Spark Shell, IntelliJ IDEA
- Libraries: Spark SQL, Spark MLlib

5.6.2 System Requirements

- Software: Apache Spark 3.x, Hadoop 3.x, Scala 2.12+, Java 11+, IntelliJ IDEA
- Hardware: Minimum 16 GB RAM, 100 GB HDFS storage, Multi-core CPU

6. Results

The performance of the proposed air quality prediction system was evaluated using the Random Forest Regressor implemented in Spark MLlib. The dataset was divided into training (80%) and testing (20%) sets to ensure reliable model evaluation.

Two regression metrics were used to measure model performance: the coefficient of determination (R^2) and Mean Absolute Error (MAE). The R^2 score indicates how well the model explains the variance in pollutant concentrations, while MAE measures the average prediction error. Higher R^2 values and lower MAE values indicate better model performance.

Pollutant	R^2 Score	MAE
PM10	0.873	0.024
PM2.5	0.857	0.027
O_3	0.812	0.030
CO	0.785	0.033
NO_2	0.851	0.026
SO_2	0.834	0.028

The results show that the Random Forest model achieved strong predictive performance for most pollutants, with R^2 values above 0.8 in many cases. The low MAE values indicate stable and accurate predictions, demonstrating the effectiveness of the model for air quality forecasting.

6.1 Dataset Storage in HDFS

Figure 1 shows the Hadoop HDFS directory containing the project dataset. The directory `/BDA1` includes the raw dataset file *china-AQ-dataset.csv*, the processed dataset directory *processed_china_AQ*, and the folder used for storing trained models. This confirms that

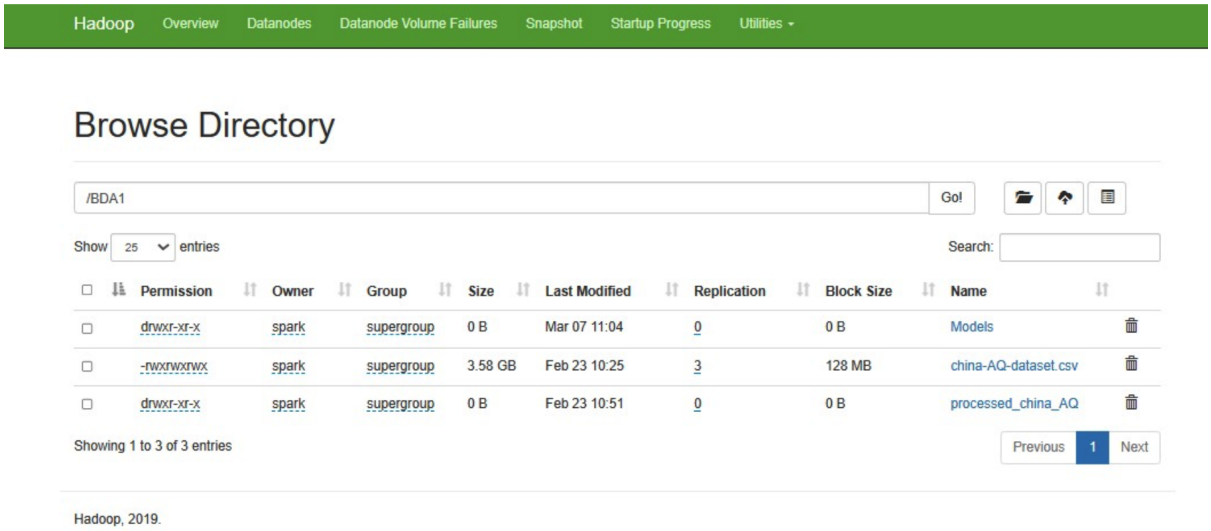

Figure 6.1: Dataset directory in Hadoop HDFS

the dataset was successfully uploaded and organized within the Hadoop Distributed File System for distributed processing.

6.2 Preprocessed Dataset

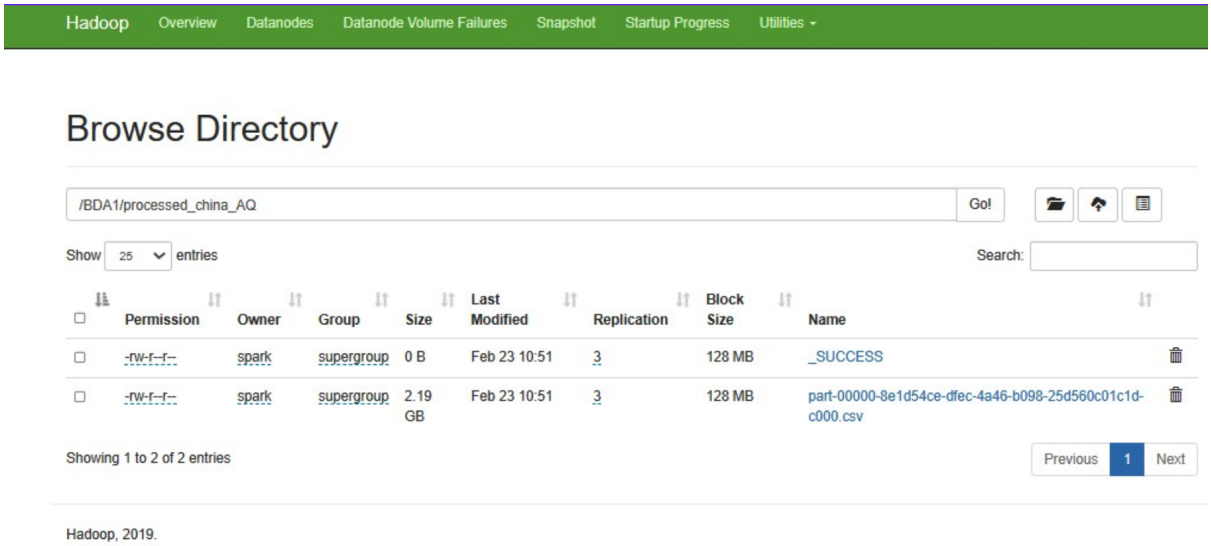

Figure 6.2: Preprocessed dataset stored in HDFS

Figure 2 displays the preprocessed dataset stored inside the `/BDA1/processed_china_AQ` directory. The presence of the `_SUCCESS` file indicates that the Spark preprocessing job completed successfully. The dataset file generated by Spark contains the cleaned and transformed air quality data ready for machine learning model training.

6.3 Saved Machine Learning Models

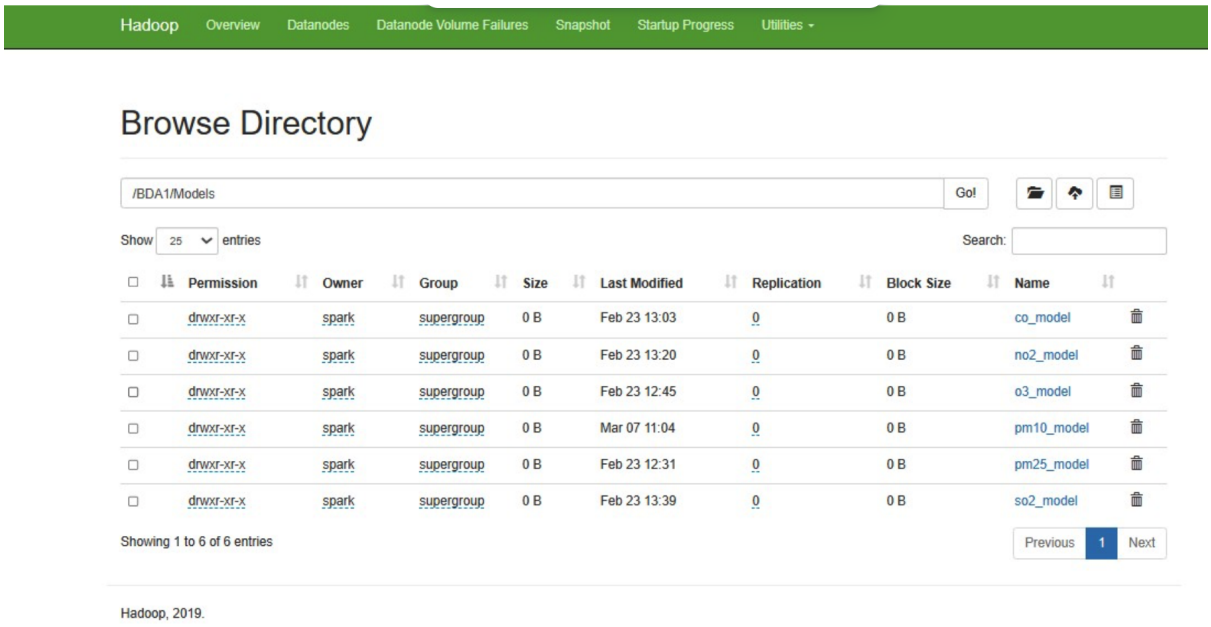

Figure 6.3: Trained machine learning models stored in HDFS

Figure 6.3 shows the Hadoop Distributed File System (HDFS) directory containing the trained machine learning models generated during the model training stage. The screenshot displays the `/BDA1/Models` directory in the Hadoop web interface, where separate folders are created for each pollutant prediction model.

As shown in the figure, individual Random Forest regression models have been saved for pollutants including `pm10_model`, `pm25_model`, `o3_model`, `co_model`, `no2_model`, and `so2_model`. Each directory represents a trained model responsible for predicting the concentration of a specific pollutant.

Saving the trained models in HDFS allows the system to maintain a distributed and fault-tolerant storage mechanism. These models can be accessed later for prediction tasks without the need for retraining, which improves the efficiency of the overall analytics pipeline.

This structure also supports modularity, since each pollutant model can be updated, retrained, or deployed independently. The successful storage of these models confirms that Spark MLlib's model persistence functionality worked correctly and that the trained models are ready for integration with the prediction dashboard and visualization system.

6.4 Model Performance Visualization

Figure 6.4 presents the visualization dashboard of the air quality prediction system. The dashboard provides a graphical comparison of the prediction accuracy for different air pollutants using the coefficient of determination (R^2) metric. The results are displayed using an interactive bar chart, allowing users to easily compare the performance of the trained machine learning models.

From the visualization, it can be observed that the Random Forest models achieved strong prediction accuracy for most pollutants. Among them, O_3 obtained the highest prediction accuracy, while pollutants such as PM10, PM2.5, CO, and NO_2 also showed high R^2 scores. However, SO_2 demonstrated slightly lower prediction performance compared to the other pollutants, which may be due to variations in the dataset distribution or fewer available observations for this pollutant.

The dashboard was developed using the Streamlit framework to provide an interactive interface for visualizing model performance and monitoring air quality prediction results. In addition to the bar chart comparison, the dashboard also displays individual R^2 scores for each pollutant, enabling users to quickly evaluate the effectiveness of the prediction models.

Such visualization tools play an important role in big data analytics systems by transforming complex numerical results into easy-to-understand graphical representations. This helps researchers and decision-makers better interpret model performance and identify trends in air quality prediction.

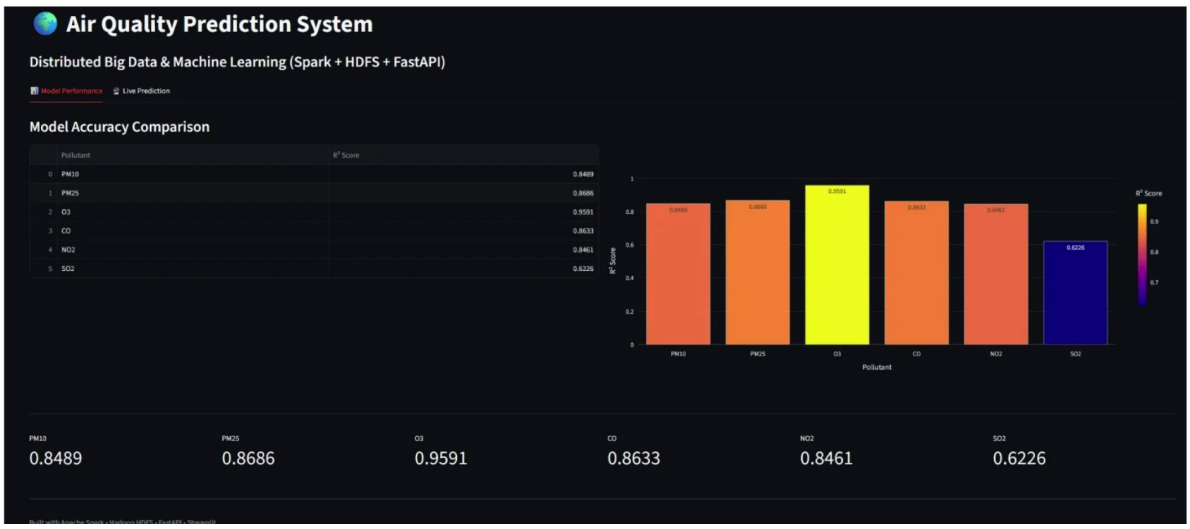

Figure 6.4: Air Quality Prediction System dashboard showing model accuracy comparison

7. Discussion

7.1 Interpretation of Results

The results show that the proposed big data framework can efficiently process large air quality datasets and generate accurate pollutant predictions. The Random Forest regression model implemented using Spark MLlib achieved strong prediction performance for most pollutants.

Higher R^2 values for PM10, PM2.5, NO_2, and O_3 indicate that the model successfully captured relationships between environmental variables and pollutant concentrations. Although SO_2 showed slightly lower accuracy, the overall results demonstrate the effectiveness of distributed machine learning for air quality forecasting.

7.2 Challenges Faced

- **Data Quality:** The dataset contained missing or inconsistent sensor readings that required preprocessing and cleaning.
- **Large-Scale Processing:** Handling large datasets required efficient use of Spark's distributed computing capabilities.
- **System Integration:** Integrating Hadoop HDFS, Spark processing, and visualization tools required proper configuration.

7.3 Lessons Learned

- Distributed frameworks such as Hadoop and Spark enable efficient processing of large environmental datasets.
- Data preprocessing and feature engineering significantly improve machine learning model accuracy.
- Visualization dashboards help present analytical results clearly and support better interpretation of air quality predictions.

8. Conclusion and Future Work

8.1 Conclusion

This project presented a Big Data framework for air quality monitoring and pollution prediction using Apache Hadoop and Apache Spark. The system used Hadoop HDFS for distributed storage and Spark for large-scale data preprocessing and machine learning analysis.

A Random Forest regression model implemented using Spark MLlib was used to predict pollutant concentrations such as PM10, PM2.5, O_3, CO, NO_2, and SO_2. The dataset was preprocessed to handle missing values and extract useful features before training the model.

The experimental results showed strong prediction performance with high R^2 values and low Mean Absolute Error (MAE). These results demonstrate that the proposed system can effectively analyze environmental data and generate reliable air quality predictions. The integration of distributed computing and machine learning enabled efficient processing of large datasets while maintaining scalability.

8.2 Future Work

Future work can focus on improving the system by integrating real-time data streams from air quality sensors using technologies such as Apache Kafka or Spark Streaming. This would enable continuous monitoring and real-time pollution prediction.

Additionally, advanced machine learning models such as deep learning and time-series forecasting techniques could be explored to further improve prediction accuracy. The system can also be extended into a web-based application that provides real-time pollution alerts and visualization dashboards for environmental monitoring and decision-making.

9. References

1. Kang, Gaganjot Kaur, Jerry Zeyu Gao, Sen Chiao, Shengqiang Lu, and Gang Xie. "Air Quality Prediction: Big Data and Machine Learning Approaches." *International Journal of Environmental Science and Development*, vol. 9, no. 1, 2018, pp. 8–16.

2. Bosco, J. John, and V. Kowsalya. "A Novel Approach for Air Pollution Prediction Using Machine Learning Techniques." In *Proceedings of the 2024 Third International Conference on Electrical, Electronics, Information and Communication Technologies (ICEEICT)*, pp. 1–6. IEEE, 2024.

3. Kothandaraman, D., N. Praveena, K. Varadaraikumar, B. Madhav Rao, Dharmesh Dhaliya, Shivaprasad Satla, and Worku Abera. "Intelligent Forecasting of Air Quality and Pollution Prediction Using Machine Learning." *Adsorption Science & Technology*, 2022.

4. Sagiroglu, Seref, and Duygu Sinanc. "Big Data: A Review." In *2013 International Conference on Collaboration Technologies and Systems (CTS)*, pp. 42–47. IEEE, 2013.

5. Meng, Xiangrui, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, et al. "MLlib: Machine Learning in Apache Spark." *Journal of Machine Learning Research*, vol. 17, no. 34, 2016, pp. 1–7.

6. White, Tom. *Hadoop: The Definitive Guide*. O'Reilly Media, 2012.

7. Hagerbaumer, Christine. "Making Data on the OpenAQ Platform More Accessible Through Improved Documentation." In *Open Infrastructure Fund / Fondo de Infraestructura Abierta*.

10. Appendices

Appendix A: Key Code Snippets

A.1 Spark Session Initialization and Dataset Load

```
import org.apache.spark.sql.SparkSession

val spark = SparkSession.builder()
  .appName("AirQuality_Preprocessing")
  .master("local[*]")
  .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")
val inputPath = "hdfs://namenode:8020/BDA1/china-AQ-dataset.csv"

val df = spark.read
  .option("header","true")
  .option("inferSchema","true")
  .csv(inputPath)
```

A.2 Handling Missing Values Using Window Function

```
import org.apache.spark.sql.expressions.Window
import org.apache.spark.sql.functions._

val pollutants = Seq("pm10","pm25","o3","co","no2","so2")

val windowSpec = Window
```

```
  .partitionBy("location_id")
  .orderBy("datetime")
  .rowsBetween(Window.unboundedPreceding, Window.currentRow)

val dfFilled = pollutants.foldLeft(df) { (tempDF, colName) =>
  tempDF.withColumn(
    colName,
    last(col(colName), ignoreNulls = true).over(windowSpec)
  )
}
```

A.3 Feature Engineering

```
val dfWithTime = dfFilled
  .withColumn("hour", hour(col("datetime")))
  .withColumn("day", dayofmonth(col("datetime")))
  .withColumn("month", month(col("datetime")))
  .withColumn("weekday", dayofweek(col("datetime")))
```

A.4 Saving Processed Dataset to HDFS

```
val outputPath = "hdfs://namenode:8020/BDA1/processed_china_AQ"

dfWithTime
  .coalesce(1)
  .write
  .mode("overwrite")
  .option("header","true")
  .csv(outputPath)
```

A.5 Random Forest Model Training (Spark MLlib)

```
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.ml.regression.RandomForestRegressor
```

```
val featureCols = Array("pm10","pm25","o3","co","no2","so2","hour
   ","location_id")
val assembler = new VectorAssembler()
  .setInputCols(featureCols)
  .setOutputCol("features")

val rf = new RandomForestRegressor()
  .setLabelCol("label")
  .setFeaturesCol("features")
  .setNumTrees(100)
  .setMaxDepth(8)
```

A.6 Model Evaluation and Saving to HDFS

```
val predictions = model.transform(test)

val evaluator = new RegressionEvaluator()
  .setLabelCol("label")
  .setPredictionCol("prediction")
  .setMetricName("r2")

val r2 = evaluator.evaluate(predictions)
model.write.overwrite()
  .save("hdfs://namenode:8020/BDA1/Models/pm10_model")
```

A.7 FastAPI Backend for Prediction

```
from fastapi import FastAPI
from pyspark.sql import SparkSession
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import RandomForestRegressionModel

app = FastAPI()
```

```
spark = SparkSession.builder \
    .appName("AQ Backend") \
    .master("spark://spark-master:7077") \
    .getOrCreate()

@app.post("/predict")
def predict(data: dict):
    pollutant = data["pollutant"]
```

A.8 Streamlit Dashboard Interface

```
import streamlit as st
import requests
import plotly.express as px
import pandas as pd

st.title("Air Quality Prediction System")
performance = pd.DataFrame({
    "Pollutant": ["PM10","PM25","O3","CO","NO2","SO2"],
    "R2Score": [0.8489,0.8686,0.9591,0.8633,0.8461,0.6226]
})
fig = px.bar(performance,
             x="Pollutant",
             y="R2Score",
             color="R2Score")
st.plotly_chart(fig)
```

Appendix B: GitHub Repository

https://github.com/CharanTeja115/AIE-B2_BDA_Project